Chromatic Rotations on a 101×101 Grid: A Complete Structural Analysis, Exact Enumeration, and Exhaustive Computational Verification

Akaash Dash
akaash.dash@gmail.com

Abstract

We present a self-contained, fully verified solution to the *chromatic rotations* combinatorial problem. A 101×101 grid of unit cells is coloured with three colours subject to two constraints: (i) no two edge-adjacent cells share a colour, and (ii) every 2×2 sub-square contains all three colours. Two such *proper* colourings are declared equivalent if one can be obtained from the other by a rotation of the grid by a multiple of 90°. We determine the exact number of equivalence classes.

Our main theoretical contribution is the **Separability Theorem**: every proper colouring of any $n \times n$ grid is uniquely determined by a base colour, a sequence of $n-1$ binary "horizontal slopes", and a sequence of $n-1$ binary "vertical slopes", all chosen independently from $\{1, 2\} \subset \mathbb{F}_3$. This yields $3 \times 2^{2(n-1)}$ proper colourings in total. Applying Burnside's Lemma with the cyclic rotation group $\mathbb{Z}_4$, we compute the fixed-point sets for each of the four rotations by solving the functional equations they impose on the separable parametrisation. For $n = 101$ the final answer is

$$\boxed{3\left(2^{198} + 2^{98} + 2^{49}\right) \;\approx\; 1.205 \times 10^{60}.}$$

Every step is independently verified by exhaustive computation for all $n \in \{2, 3, \ldots, 11\}$ using a backtracking search with early pruning, parallelised across all CPU cores. The formula for odd n is confirmed at $n = 3, 5, 7, 9, 11$; the total-count formula is confirmed for all $n \leq 11$. The entire verification (all ten grid sizes) completes in under 30 seconds on a laptop CPU. Additionally, the complete proof chain — from grid definitions through the Separability Theorem, Burnside fixed-point analysis, and closed-form formula — has been machine-checked in **Lean 4** with the MATHLIB library.

1 Introduction

1.1 Motivation and context

Counting combinatorial objects up to the action of a symmetry group is a classical and important problem in combinatorics. The standard tool is Burnside's Lemma (sometimes called the Cauchy–Frobenius Lemma), which reduces the problem to counting objects fixed by each group element separately. The difficulty typically lies in the fixed-point analysis.

The present problem combines three familiar ingredients — proper graph colouring, a local density constraint, and rotational symmetry — in a way that turns out to force remarkable global structure. The key discovery is that both constraints together force the colourings to be *separable*: the colour at cell (r, c) is determined entirely by a row-offset and a column-offset that are independent of each other. This makes the total count and the Burnside analysis tractable.

1.2 Problem statement

We work with a square grid $\mathcal{G}$ of $n = 101$ rows and $n = 101$ columns of unit cells, indexed $(r, c) \in \{0, 1, \ldots, 100\}^2$. Cells are coloured with three colours encoded as elements of $\mathbb{F}_3 = \{0, 1, 2\}$ (concretely: yellow $= 0$, grey $= 1$, green $= 2$, though the labelling plays no role).

Definition 1.1 (Proper colouring)**.** *A function $f : \{0, \ldots, 100\}^2 \to \{0, 1, 2\}$ is a* proper colouring *of $\mathcal{G}$ if:*

1. ***Adjacency condition.*** *No two horizontally or vertically adjacent cells share a colour:*
$$f(r, c) \neq f(r, c + 1) \quad \textit{and} \quad f(r, c) \neq f(r + 1, c)$$
for all r, c such that the indices are in range.

2. ***Richness condition.*** *Every 2×2 sub-square contains all three colours:*
$$\big\{f(r, c),\ f(r, c + 1),\ f(r + 1, c),\ f(r + 1, c + 1)\big\} = \{0, 1, 2\}$$
for all $(r, c) \in \{0, \ldots, 99\}^2$.

Definition 1.2 (Rotation and equivalence)**.** *Let ρ denote the 90° clockwise rotation of the grid. Its action on a colouring f is*
$$(\rho \cdot f)(r, c) \ = \ f(100 - c,\ r).$$
Two proper colourings f and g are rotationally equivalent *if $g = \rho^k \cdot f$ for some $k \in \{0, 1, 2, 3\}$. The equivalence classes under this relation are the* groups *referred to in the problem.*

The four rotations and their coordinate maps are:

$$\rho^0 : (r, c) \mapsto (r, c), \qquad \rho^1 : (r, c) \mapsto (c, 100 - r),$$
$$\rho^2 : (r, c) \mapsto (100 - r, 100 - c), \qquad \rho^3 : (r, c) \mapsto (100 - c, r).$$

The main question is: how many rotationally distinct groups of proper colourings of the 101×101 grid are there?

1.3 Organisation of the paper

Section 2 analyses the local structure of a 2×2 sub-square in a proper colouring, introducing slope notation and deriving the fundamental propagation rules. Section 3 states and proves the Separability Theorem. Section 4 counts all proper colourings. Section 5 applies Burnside's Lemma and computes all fixed-point sets. Section 6 collects the final answer. Section 7 describes the exhaustive computational verification (extended to $n \leq 11$) and presents the algorithm and its output. Section 8 discusses generalisations.

2 Local Structure of Proper Colourings

2.1 The diagonal principle

Consider a 2×2 sub-square at position (r, c):

$$\begin{pmatrix} a & b \\ c' & d \end{pmatrix} = \begin{pmatrix} f(r, c) & f(r, c + 1) \\ f(r + 1, c) & f(r + 1, c + 1) \end{pmatrix}.$$

(We use c' for $f(r + 1, c)$ to avoid clash with the column index c.)

The adjacency condition says $a \neq b$, $a \neq c'$, $b \neq d$, $c' \neq d$. The richness condition says the multiset $\{a, b, c', d\}$ contains all three elements of $\mathbb{F}_3$.

Lemma 2.1 (Diagonal Principle)**.** *In any proper colouring, for every 2×2 sub-square, exactly one of the following holds:*

- ***Case A*** *(main diagonal): $f(r, c) = f(r + 1, c + 1)$, i.e. $a = d$, and $\{b, c'\}$ are the two remaining colours, necessarily distinct.*

- ***Case B** (anti-diagonal): $f(r, c+1) = f(r+1, c)$, i.e. $b = c'$, and $\{a, d\}$ are the two remaining colours, necessarily distinct.*

Proof. Since we have four cells, three colours, and all three must appear, exactly one colour appears twice. The adjacency condition rules out any two adjacent cells sharing a colour; the only non-adjacent pairs in a 2×2 square are the two diagonals, (a, d) and (b, c'). Exactly one diagonal pair shares a colour. □

2.2 Slope notation

We encode the colour differences between adjacent cells as *slopes*, which live in $\{1, 2\} = \mathbb{F}_3 \setminus \{0\}$.

Definition 2.2 (Slopes)**.** *For a proper colouring f, define:*

- Horizontal slope: $h_{r,c} := f(r, c+1) - f(r, c) \pmod 3 \in \{1, 2\}$, *for* $r \in \{0, \ldots, 100\}$ *and* $c \in \{0, \ldots, 99\}$.
- Vertical slope: $v_{r,c} := f(r+1, c) - f(r, c) \pmod 3 \in \{1, 2\}$, *for* $r \in \{0, \ldots, 99\}$ *and* $c \in \{0, \ldots, 100\}$.

We write $\bar{x}$ for the complement of $x \in \{1, 2\}$, i.e. $\bar{1} = 2$ and $\bar{2} = 1$. Note $\bar{x} \equiv -x \pmod 3$.

In slope notation the four corners of the sub-square at (r, c) are:

$$a, \quad b = a + h_{r,c}, \quad c' = a + v_{r,c}, \quad d = a + v_{r,c} + h_{r+1,c}.$$

2.3 Case analysis and slope propagation

We now determine what each case of Lemma 2.1 implies for the slopes.

2.3.1 Case A: $a = d$

The condition $a = d$ reads $v_{r,c} + h_{r+1,c} \equiv 0 \pmod 3$, giving

$$h_{r+1,c} = \bar{v_{r,c}}. \tag{1}$$

Similarly, $d - b = a - (a + h_{r,c}) = -h_{r,c}$, i.e. $f(r+1, c+1) - f(r, c+1) = -h_{r,c}$, giving

$$v_{r,c+1} = \bar{h_{r,c}}. \tag{2}$$

For all three colours to appear we need $b \neq c'$, i.e. $h_{r,c} \neq v_{r,c}$. Since both are in $\{1, 2\}$, this means $\{h_{r,c}, v_{r,c}\} = \{1, 2\}$, so $h_{r,c} = \bar{v_{r,c}}$. Substituting into (1)–(2):

$$\boxed{h_{r+1,c} = \bar{v_{r,c}} = h_{r,c}, \quad v_{r,c+1} = \bar{h_{r,c}} = v_{r,c}.} \tag{3}$$

2.3.2 Case B: $b = c'$

The condition $b = c'$ reads $h_{r,c} = v_{r,c}$, and the three distinct values are a, $a + h_{r,c}$ (appearing twice), and the third colour which must be $a + 2h_{r,c} = a - h_{r,c} \pmod 3$. So $d = a - h_{r,c}$. Computing:

$$h_{r+1,c} = d - c' = (a - h_{r,c}) - (a + h_{r,c}) = -2h_{r,c} \equiv h_{r,c} \pmod 3, \tag{4}$$

$$v_{r,c+1} = d - b = (a - h_{r,c}) - (a + h_{r,c}) = -2h_{r,c} \equiv h_{r,c} = v_{r,c} \pmod 3. \tag{5}$$

So:

$$\boxed{h_{r+1,c} = h_{r,c}, \quad v_{r,c+1} = v_{r,c} = h_{r,c}.} \tag{6}$$

2.4 The fundamental invariance

Equations (3) and (6) both yield the same conclusion:

Proposition 2.3 (Slope Invariance)**.** *In any proper colouring, for every (r, c) with both indices in range:*

$$h_{r+1,c} = h_{r,c} \quad \textit{and} \quad v_{r,c+1} = v_{r,c}.$$

Proof. In Case A we showed $h_{r+1,c} = v_{r,c}^-$. Since Case A requires $h_{r,c} = v_{r,c}^-$, we get $h_{r+1,c} = h_{r,c}$. Similarly $v_{r,c+1} = h_{r,c}^- = v_{r,c}$.

In Case B: $h_{r+1,c} = h_{r,c}$ from (4) and $v_{r,c+1} = v_{r,c}$ from (5). □

By induction on r and c:

Corollary 2.4. *For any proper colouring f of any $n \times n$ grid:*

- *The horizontal slope $h_{r,c}$ is independent of r: define $h_c := h_{r,c}$ for any r.*
- *The vertical slope $v_{r,c}$ is independent of c: define $v_r := v_{r,c}$ for any c.*

This corollary is the heart of the entire argument. The slope at every horizontal edge in a given column is the same, and the slope at every vertical edge in a given row is the same.

3 The Separability Theorem

Theorem 3.1 (Separability)**.** *Let $n \geq 2$. A function $f : \{0, \ldots, n-1\}^2 \to \mathbb{F}_3$ is a proper colouring of the $n \times n$ grid if and only if it has the form*

$$f(r, c) = \text{base} + R(c) + D(r) \pmod 3, \tag{7}$$

where:

- $\text{base} \in \{0, 1, 2\}$,
- $R : \{0, \ldots, n-1\} \to \mathbb{F}_3$ *with* $R(0) = 0$ *and* $R(c) - R(c-1) \in \{1, 2\}$ *for all* $c \geq 1$ *(equivalently,* $R(c) = \sum_{j=0}^{c-1} h_j \pmod 3$ *with each* $h_j \in \{1, 2\}$*),*
- $D : \{0, \ldots, n-1\} \to \mathbb{F}_3$ *with* $D(0) = 0$ *and* $D(r) - D(r-1) \in \{1, 2\}$ *for all* $r \geq 1$ *(equivalently,* $D(r) = \sum_{i=0}^{r-1} v_i \pmod 3$ *with each* $v_i \in \{1, 2\}$*).*

Furthermore, the triple $(\text{base}, (h_j), (v_i))$ is uniquely determined by f, and different triples give different colourings.

Proof. **Necessity.** By Corollary 2.4, every proper colouring has well-defined column slopes $h_c \in \{1, 2\}$ and row slopes $v_r \in \{1, 2\}$. Define R and D by the given recurrences and $\text{base} = f(0, 0)$. By telescoping:

$$f(r, c) = f(0, 0) + \big[f(0, c) - f(0, 0)\big] + \big[f(r, 0) - f(0, 0)\big] = \text{base} + R(c) + D(r),$$

where we used $f(0, c) = f(0, 0) + \sum_{j=0}^{c-1} h_j = \text{base} + R(c)$ and $f(r, 0) = f(0, 0) + \sum_{i=0}^{r-1} v_i = \text{base} + D(r)$. All equalities are modulo 3.

Sufficiency. Given any valid triple $(\text{base}, (h_j), (v_i))$, define $f(r, c) = \text{base} + R(c) + D(r) \pmod 3$.

Adjacency: $f(r, c+1) - f(r, c) = R(c+1) - R(c) = h_c \in \{1, 2\} \neq 0$. Similarly $f(r+1, c) - f(r, c) = v_r \neq 0$. ✓

Richness: The four values of the sub-square at (r, c) are $\{a + 0,\ a + h_c,\ a + v_r,\ a + h_c + v_r\}$ where $a = f(r, c)$. We need these to be all three colours. Since $h_c, v_r \in \{1, 2\}$:

- If $h_c = v_r$: the four values are $a,\ a + h_c,\ a + h_c,\ a + 2h_c$. Since $2h_c = -h_c \neq h_c$ and $\{0, h_c, 2h_c\}$ covers all of $\mathbb{F}_3$ (as $h_c \in \{1, 2\}$ generates the group), all three colours appear. ✓
- If $h_c \neq v_r$: then $\{h_c, v_r\} = \{1, 2\}$ and $h_c + v_r = 3 \equiv 0 \pmod 3$. The four values are $a,\ a + h_c,\ a + v_r,\ a + 0$, i.e. a appears twice, and $a + 1, a + 2$ also appear. All three colours appear. ✓

Uniqueness and bijectivity. Given f, the parameters are uniquely recovered as $\text{base} = f(0, 0)$, $h_c = f(0, c+1) - f(0, c) \pmod 3$, and $v_r = f(r+1, 0) - f(r, 0) \pmod 3$. Injectivity follows. □

Remark 3.2. *The separability result is striking. The richness condition is* a priori *a nonlinear coupling between all four cells of every 2×2 block. Yet together with the adjacency condition it forces the entire colouring to have an additive structure in which rows and columns are completely independent.*

4 Counting Proper Colourings

Theorem 4.1 (Total count)**.** *For any $n \geq 2$, the total number of proper colourings of the $n \times n$ grid is*

$$|\mathcal{P}_n| = 3 \times 2^{2(n-1)}.$$

Proof. By Theorem 3.1, proper colourings biject with triples $\big(\text{base},\ (h_0, \ldots, h_{n-2}),\ (v_0, \ldots, v_{n-2})\big)$ where $\text{base} \in \{0, 1, 2\}$ and all $h_j, v_i \in \{1, 2\}$. The count is $3 \times 2^{n-1} \times 2^{n-1} = 3 \times 2^{2(n-1)}$. □

For $n = 101$ this gives $|\mathcal{P}_{101}| = 3 \times 2^{200}$.

Verified cases. Table 1 shows the formula against the exhaustive backtracking count for all $n \leq 11$. All entries agree.

Table 1: Total proper colourings for $n = 2$–11, verified by backtracking search.

n	Formula $3 \times 2^{2(n-1)}$	Verified count	Match
2	12	12	✓
3	48	48	✓
4	192	192	✓
5	768	768	✓
6	3 072	3 072	✓
7	12 288	12 288	✓
8	49 152	49 152	✓
9	196 608	196 608	✓
10	786 432	786 432	✓
11	3 145 728	3 145 728	✓

5 Burnside's Lemma and Fixed-Point Analysis

5.1 Setup

The symmetry group acting on proper colourings is the cyclic group of rotations $\mathbb{Z}_4 = \{e, \rho, \rho^2, \rho^3\}$. By Burnside's Lemma, the number of rotationally distinct equivalence classes is

$$|\mathcal{C}| = \frac{1}{4} \sum_{k=0}^{3} |\text{Fix}(\rho^k)|, \tag{8}$$

where $\text{Fix}(\rho^k) = \{f \in \mathcal{P} : \rho^k \cdot f = f\}$.

We compute $|\text{Fix}(\rho^k)|$ for each k by translating the fixed-point condition into constraints on the separable parametrisation $(\text{base}, (h_c), (v_r))$.

5.2 Identity: $|\text{Fix}(\rho^0)|$

Every proper colouring is fixed by the identity:

$$|\text{Fix}(\rho^0)| = |\mathcal{P}_{101}| = 3 \times 2^{200}.$$

5.3 180° rotation: $|\mathrm{Fix}(\rho^2)|$

The 180° rotation maps $(r, c) \mapsto (100 - r, 100 - c)$. A colouring f is fixed iff $f(r, c) = f(100 - r, 100 - c)$ for all (r, c). In the separable parametrisation this becomes:

$$\text{base} + R(c) + D(r) \equiv \text{base} + R(100 - c) + D(100 - r) \pmod 3,$$

i.e.

$$R(c) + D(r) = R(100 - c) + D(100 - r) \quad \forall r, c. \tag{9}$$

Step 1. Set $r = 0$ (using $D(0) = 0$):

$$R(c) = R(100 - c) + D(100) \quad \forall c. \tag{$\star$}$$

Step 2. Set $c = 0$ (using $R(0) = 0$):

$$D(r) = D(100 - r) + D(100) \quad \forall r. \tag{$\star\star$}$$

Step 3. Set $r = c = 0$: $0 = 0 + D(100)$, so $D(100) = 0$.

With $D(100) = 0$, equations ($\star$) and ($\star\star$) simplify to:

$$R(c) = R(100 - c) \quad \text{and} \quad D(r) = D(100 - r).$$

Step 4. Differencing the condition $R(c) = R(100 - c)$ between consecutive values of c:

$$R(c) - R(c - 1) = R(100 - c + 1) - R(100 - c), \quad \text{i.e.,} \quad h_{c-1} = -(h_{100-c-1}) \pmod 3,$$

or equivalently $h_{c-1} + h_{100-c-1} \equiv 0 \pmod 3$, i.e. $h_{c-1} = \bar{h}_{99-c+1}$. For $c = 1, \ldots, 50$:

$$h_j = \bar{h}_{99-j}, \quad j = 0, \ldots, 49. \tag{10}$$

So the 100 slopes $(h_0, \ldots, h_{99})$ are partitioned into 50 anti-palindrome pairs, each pair of the form $\{h_j, \bar{h}_j\}$, giving 50 free binary choices.

Similarly $v_i = \bar{v}_{99-i}$ for $i = 0, \ldots, 49$, giving 50 more free binary choices.

Step 5. Verify the global-sum condition $D(100) = \sum_{i=0}^{99} v_i \equiv 0$:

$$\sum_{i=0}^{99} v_i = \sum_{i=0}^{49} (v_i + v_{99-i}) = \sum_{i=0}^{49} (v_i + \bar{v}_i) = \sum_{i=0}^{49} 3 = 0 \pmod 3.$$

Automatically satisfied. ✓

Conclusion.

$$|\mathrm{Fix}(\rho^2)| = 3 \times 2^{50} \times 2^{50} = 3 \times 2^{100}.$$

5.4 90° rotation: $|\mathrm{Fix}(\rho^1)|$

The 90° clockwise rotation maps $(r, c) \mapsto (100 - c, r)$. A colouring f is fixed iff $f(r, c) = f(100 - c, r)$ for all (r, c). In the separable parametrisation:

$$R(c) + D(r) = R(r) + D(100 - c) \quad \forall r, c. \tag{11}$$

Step 1. The right-hand side can be written as $R(r) + D(100 - c)$. The quantity $R(c) + D(r) - (R(r) + D(100 - c))$ must be zero for all r, c. Rearranging:

$$[R(c) - D(100 - c)] = [R(r) - D(r)] \quad \forall r, c.$$

Both sides are functions of a single variable; the only way the equation holds for all r and all c is if both are equal to the same constant K:

$$R(c) = D(100 - c) + K \quad \text{and} \quad D(r) = R(r) + K.$$

Step 2. Setting $c = 0$: $R(0) = D(100) + K$, i.e. $0 = D(100) + K$, so $K = -D(100)$. Setting $r = 0$: $D(0) = R(0) + K$, i.e. $0 = 0 + K$, so $K = 0$. Therefore $D(100) = 0$ and:

$$D(r) = R(r) \quad \forall r, \quad \text{i.e.,} \quad v_i = h_i \quad \forall i. \tag{12}$$

Step 3. Also $R(c) = D(100-c) = R(100-c)$, giving the palindrome condition $R(c) = R(100-c)$. Differencing as before:

$$h_j = h_{99-j}, \quad j = 0, \ldots, 49. \tag{13}$$

(Identical to the 180° condition on h; see (10).)

Step 4. Verify $D(100) = 0$: Same calculation as in Section 5.3. ✓

Step 5. Verify consistency with (11): Substituting $D(r) = R(r)$ and $R(c) = R(100 - c)$, the condition becomes $R(c) + R(r) = R(r) + D(100 - c) = R(r) + R(100 - c) = R(r) + R(c)$. ✓

Degrees of freedom. The free parameters are $h_0, \ldots, h_{49} \in \{1, 2\}$ (50 binary choices); then $h_{99-j} = h_j$ determines the remaining 50 horizontal slopes, and $v_i = h_i$ determines all vertical slopes.

Conclusion.

$$|\mathrm{Fix}(\rho^1)| = 3 \times 2^{50}.$$

5.5 270° rotation: $|\mathrm{Fix}(\rho^3)|$

The 270° clockwise rotation maps $(r, c) \mapsto (c, 100 - r)$. A colouring f is fixed iff $f(r, c) = f(c, 100 - r)$. In the separable parametrisation:

$$R(c) + D(r) = R(100 - r) + D(c) \quad \forall r, c.$$

Rearranging: $[R(c) - D(c)] = [R(100 - r) - D(r)]$ for all r, c. By the same constant argument: both sides equal K, so $R(c) = D(c) + K$ and $R(100 - r) = D(r) + K$.

Setting $c = 0$: $0 = D(0) + K = K$. So $K = 0$, $R(c) = D(c)$ for all c (i.e. $h_i = v_i$), and $R(100 - r) = D(r)$ for all r (i.e. $R(100 - r) = R(r)$, the same anti-palindrome condition). The analysis is identical to Section 5.4 with the roles of (r, c) relabelled.

Conclusion.

$$|\mathrm{Fix}(\rho^3)| = 3 \times 2^{50}.$$

5.6 Summary of fixed-point counts

Table 2: Fixed-point counts under each rotation. Odd-n rows include the closed-form formula; even-n rows are ground-truth only (no formula derived).

	$\lvert\mathrm{Fix}(\rho^k)\rvert$				Burnside	Formula
Grid	$k=0$	$k=1$	$k=2$	$k=3$	groups	groups
2×2	12	0	0	0	3	–
3×3	48	6	12	6	18	18
4×4	192	0	0	0	48	–
5×5	768	12	48	12	210	210
6×6	3072	0	0	0	768	–
7×7	12288	24	192	24	3132	3132
8×8	49152	0	0	0	12288	–
9×9	196608	48	768	48	49368	49368
10×10	786432	0	0	0	196608	–
11×11	3145728	96	3072	96	787248	787248
101×101	$3 \cdot 2^{200}$	$3 \cdot 2^{50}$	$3 \cdot 2^{100}$	$3 \cdot 2^{50}$	–	$3(2^{198} + 2^{98} + 2^{49})$

Note on even n. For even n, the brute-force shows $|\mathrm{Fix}(\rho^1)| = |\mathrm{Fix}(\rho^2)| = |\mathrm{Fix}(\rho^3)| = 0$. Intuitively, a 90° rotation of an even-n grid has no fixed cells at all (there is no centre cell), making it impossible for a proper colouring (which requires distinct adjacent colours) to be invariant. For odd n the centre cell is fixed by all rotations, allowing non-trivial fixed colourings.

6 The Final Answer

Theorem 6.1 (Main result)**.** *The number of rotationally distinct groups of proper colourings of the* 101×101 *grid is*

$$|\mathcal{C}| = 3\big(2^{198} + 2^{98} + 2^{49}\big)\,.$$

Proof. By Burnside's Lemma (8) and the fixed-point counts computed in Section 5:

$$\begin{aligned}|\mathcal{C}| &= \frac{1}{4}\Big(3 \cdot 2^{200} + 3 \cdot 2^{50} + 3 \cdot 2^{100} + 3 \cdot 2^{50}\Big) \\ &= \frac{3}{4}\Big(2^{200} + 2^{100} + 2 \cdot 2^{50}\Big) \\ &= \frac{3}{4}\Big(2^{200} + 2^{100} + 2^{51}\Big).\end{aligned}$$

To verify this is an integer, factor out $4 = 2^2$:

$$2^{200} + 2^{100} + 2^{51} = 2^{51}\big(2^{149} + 2^{49} + 1\big)\,,$$

which is divisible by $4 = 2^2$ since $51 \geq 2$. Therefore:

$$|\mathcal{C}| = \frac{3 \cdot 2^{51}(2^{149} + 2^{49} + 1)}{4} = 3 \cdot 2^{49}(2^{149} + 2^{49} + 1) = 3(2^{198} + 2^{98} + 2^{49}).$$

□

Numerical value.

$$\begin{aligned}3(2^{198} + 2^{98} + 2^{49}) &\approx 3 \times 4.018 \times 10^{59} \\ &= 1.205 \times 10^{60}.\end{aligned}$$

The exact 60-digit decimal is 120520353319424270665647156925682268984182341907706701414400 0.

7 Computational Verification

7.1 Algorithm

Flat enumeration of all 3^{n^2} candidates becomes infeasible for $n \geq 5$ ($3^{25} \approx 8.5 \times 10^{11}$ for $n = 5$). We therefore use *backtracking with early pruning*.

Key observation. Once the first row and the first column of the grid are fixed, every remaining cell is *uniquely determined* by the 2×2 richness constraint. Concretely, given the three known corners of the 2×2 square above and to the left of cell (r, c) (with $r, c \geq 1$), the richness condition forces the fourth corner to be the unique missing colour. (This is easy to verify: with adjacency, the three known cells always cover two or three distinct colours, and in every case exactly one colour for the new cell satisfies both adjacency and richness.)

This means the search tree has at most $3 \times 2^{(n-1)} \times 2^{(n-1)} = 3 \times 2^{2(n-1)}$ leaves — exactly the number of valid colourings — with no wasted backtracking. For $n = 11$ this is $3\,145\,728$ leaves versus 5.4×10^{62} flat candidates.

Parallelisation. We generate all $3 \times 2^{n-1}$ valid first-row configurations (sequences with no two adjacent entries equal) and dispatch each as an independent job to a pool of worker processes. For $n = 11$ this creates 3072 jobs distributed across 8 CPU cores. Each worker runs an iterative (non-recursive) backtracking loop from its fixed first row.

Memory strategy. For $n \leq 7$ each worker returns the list of proper colourings it finds (at most 4096 per job), and the main process performs full set-equality and orbit-counting verification. For $n \geq 8$, where collecting all colorings would require gigabytes, each worker instead returns only aggregate counts: total found, separability-check count, and the four Burnside fixed-point counts. The main process sums these to produce the final verification figures.

7.2 Core implementation

Listing 1 gives the key functions. The complete implementation (including the parallel dispatcher and logging infrastructure) is in the accompanying file `main.py`.

```python
import itertools, multiprocessing as mp, os
from tqdm import tqdm

NUM_WORKERS = os.cpu_count()  # 8 on Apple M1 Pro

# -- constraint checkers ---------------------------------------

def is_proper(g, n):
    for r in range(n):
        for c in range(n):
            v = g[r*n+c]
            if c+1 < n and v == g[r*n+c+1]: return False
            if r+1 < n and v == g[(r+1)*n+c]: return False
    for r in range(n-1):
        for c in range(n-1):
            sq = {g[r*n+c], g[r*n+c+1],
                  g[(r+1)*n+c], g[(r+1)*n+c+1]}
            if len(sq) < 3: return False
    return True

def rotate90(g, n):
    out = [0]*(n*n)
    for r in range(n):
        for c in range(n):
            out[c*n+(n-1-r)] = g[r*n+c]
    return tuple(out)

def is_separable(g, n):
    base = g[0]
    R = [(g[c]-base)%3 for c in range(n)]
    D = [(g[r*n]-base)%3 for r in range(n)]
    return all(g[r*n+c]==(base+R[c]+D[r])%3
               for r in range(n) for c in range(n))

def generate_separable(n):
    out = []
    for base in range(3):
        for hs in itertools.product((1,2), repeat=n-1):
            R = [0]*n
            for c in range(1,n): R[c]=(R[c-1]+hs[c-1])%3
            for vs in itertools.product((1,2), repeat=n-1):
                D = [0]*n
                for r in range(1,n): D[r]=(D[r-1]+vs[r-1])%3
                g = tuple((base+R[c]+D[r])%3
                          for r in range(n) for c in range(n))
                out.append(g)
    return out

def burnside_groups(proper, n):
    fix = [0,0,0,0]; seen = set(); groups = 0
    for g in proper:
        fix[0] += 1
        g1=rotate90(g,n); g2=rotate90(g1,n); g3=rotate90(g2,n)
        if g==g1: fix[1]+=1
        if g==g2: fix[2]+=1
        if g==g3: fix[3]+=1
        if g not in seen:
            groups += 1; seen |= {g,g1,g2,g3}
    assert sum(fix)//4 == groups
    return groups, fix

# -- closed-form formulas (odd n) ------------------------------

def formula_groups_odd(n):
    k = n-1
    return (3*(2**(2*k) + 2**(k//2+1) + 2**k))//4
```

```
67
68  def formula_fix_odd(n):
69      k = n-1
70      fe=3*2**(2*k); fr90=3*2**(k//2); fr180=3*2**k
71      return fe, fr90, fr180, fr90
72
73  # -- iterative backtracking worker (module-level for pickling) -
74
75  def _worker(args):
76      """
77      Iterative DFS from a fixed first-row prefix.
78
79      collect_mode=True  -> return list of proper colorings
80      collect_mode=False -> return (total, sep, r90, r180, r270)
81      """
82      prefix, n, collect_mode = args
83      nn = n*n
84      grid = list(prefix) + [0]*(nn-len(prefix))
85      start = len(prefix)
86      nv = [0]*(nn+1)    # next color to try at each position
87      proper = [] if collect_mode else None
88      total=sep=r90=r180=r270=0
89
90      pos = start
91      while pos >= start:
92          if pos == nn:
93              if collect_mode:
94                  proper.append(tuple(grid))
95              else:
96                  g=tuple(grid); total+=1
97                  if is_separable(g,n): sep+=1
98                  g1=rotate90(g,n)
99                  if g==g1: r90+=1
100                 g2=rotate90(g1,n)
101                 if g==g2: r180+=1
102                 if g==rotate90(g2,n): r270+=1
103             pos -= 1
104         else:
105             r,c = divmod(pos,n)
106             v=nv[pos]; found=False
107             while v < 3:
108                 ok=True
109                 if c>0 and v==grid[pos-1]:   ok=False
110                 if ok and r>0 and v==grid[pos-n]: ok=False
111                 if ok and r>0 and c>0:
112                     sq={grid[(r-1)*n+(c-1)],grid[(r-1)*n+c],
113                         grid[r*n+(c-1)],v}
114                     if len(sq)<3: ok=False
115                 if ok: found=True; break
116                 v+=1
117             if found:
118                 grid[pos]=v; nv[pos]=v+1; nv[pos+1]=0; pos+=1
119             else:
120                 nv[pos]=0; pos-=1
121
122     return proper if collect_mode else (total,sep,r90,r180,r270)
123
124 # -- job generation and parallel dispatch ----------------------
125
126 def _first_row_jobs(n, collect_mode):
127     jobs=[]; stack=[(v,) for v in range(3)]
128     while stack:
129         row=stack.pop()
130         if len(row)==n:
131             jobs.append((row,n,collect_mode))
132         else:
133             last=row[-1]
134             for v in range(3):
135                 if v!=last: stack.append(row+(v,))
136     return jobs
137
138 def run_parallel(n, collect_mode):
139     jobs = _first_row_jobs(n, collect_mode)
140     with mp.Pool(processes=NUM_WORKERS) as pool:
141         chunks = list(tqdm(pool.imap_unordered(_worker, jobs),
142                            total=len(jobs), leave=False))
143     if collect_mode:
144         return [g for chunk in chunks for g in chunk]
145     total=sep=r90=r180=r270=0
146     for t,s,a,b,c in chunks:
147         total+=t; sep+=s; r90+=a; r180+=b; r270+=c
```

```
    return total,sep,r90,r180,r270
```

Listing 1: Core verification functions.

7.3 Verified output

Running `uv run python main.py` produces the following (tqdm progress bars omitted):

```
  ================================================================
  CHROMATIC ROTATIONS  --  verification  n = 2 ... 11
    Workers : 8    Log : verification_output.log
  ================================================================

    n = 2   (2x2 grid, 81 candidates)
    Proper colorings :          12  (formula 3*2^2  = 12)   OK
    Separability     : ALL separable
    Separable gen    : sets equal  (12 generated)
    Fix(e,90,180,270): [12, 0, 0, 0]  ->  12 / 4 = 3 groups
    (n=2 even -- no closed-form formula)

    n = 3   (3x3 grid, 19,683 candidates)
    Proper colorings :          48  (formula 3*2^4  = 48)   OK
    Separability     : ALL separable
    Separable gen    : sets equal  (48 generated)
    Fix(e,90,180,270): [48, 6, 12, 6]  ->  72 / 4 = 18 groups
    Formula fixes    : [48, 6, 12, 6]   OK
    Formula groups   : 18               OK

    n = 4   (4x4 grid)
    Proper colorings :         192  (formula 3*2^6  = 192)  OK
    Fix(e,90,180,270): [192, 0, 0, 0]  ->  192 / 4 = 48 groups

    n = 5   (5x5 grid)
    Proper colorings :         768  (formula 3*2^8  = 768)  OK
    Fix(e,90,180,270): [768, 12, 48, 12]  ->  840 / 4 = 210 groups
    Formula fixes    : [768, 12, 48, 12]  OK
    Formula groups   : 210               OK

    n = 6   (6x6 grid)
    Proper colorings :      3,072  (formula 3*2^10 = 3,072)  OK
    Fix(e,90,180,270): [3072, 0, 0, 0]  ->  3,072 / 4 = 768 groups

    n = 7   (7x7 grid)
    Proper colorings :     12,288  (formula 3*2^12 = 12,288)  OK
    Fix(e,90,180,270): [12288, 24, 192, 24]  ->  12,528 / 4 = 3,132 groups
    Formula fixes    : [12288, 24, 192, 24]  OK
    Formula groups   : 3,132               OK

    n = 8   (8x8 grid, streamed)
    Proper colorings :     49,152  (formula 3*2^14 = 49,152)  OK
    Fix(e,90,180,270): [49152, 0, 0, 0]  ->  49,152 / 4 = 12,288 groups

    n = 9   (9x9 grid, streamed)
    Proper colorings :    196,608  (formula 3*2^16 = 196,608)  OK
    Fix(e,90,180,270): [196608, 48, 768, 48]  ->  197,472 / 4 = 49,368 groups
    Formula fixes    : [196608, 48, 768, 48]  OK
    Formula groups   : 49,368              OK

    n = 10  (10x10 grid, streamed)
    Proper colorings :    786,432  (formula 3*2^18 = 786,432)  OK
    Fix(e,90,180,270): [786432, 0, 0, 0]  ->  786,432 / 4 = 196,608 groups

    n = 11  (11x11 grid, streamed)
    Proper colorings :  3,145,728  (formula 3*2^20 = 3,145,728)  OK
    Fix(e,90,180,270): [3145728, 96, 3072, 96]  ->  3,148,992 / 4 = 787,248
    Formula fixes    : [3145728, 96, 3072, 96]  OK
    Formula groups   : 787,248              OK

    All n <= 11: every proper coloring is separable.  OK

  ================================================================
    EXTRAPOLATION TO n = 101
  ================================================================
    Fix(e)    = 3 * 2^200
    Fix(r90)  = Fix(r270) = 3 * 2^50
    Fix(r180) = 3 * 2^100
    Groups = (3*2^200 + 3*2^50 + 3*2^100 + 3*2^50) / 4
           = (3/4)(2^200 + 2^100 + 2^51)
```

```
           = 3(2^198 + 2^98 + 2^49)
    Numerical: 1205203533194242706656471569256822689841823419077067014144000
```

Listing 2: Complete verification output (n = 2–11).

7.4 What the verification proves

The computation provides a certificate for the following claims, with no possibility of error for $n \leq 11$:

1. **Separability.** Among all proper colourings found for $n \in \{2, \ldots, 11\}$, *every single one* passes the `is_separable()` test. No exceptions.
2. **Total count.** The formula $3 \times 2^{2(n-1)}$ matches the backtracking count for all ten grid sizes.
3. **Fixed-point counts (odd n).** For $n \in \{3, 5, 7, 9, 11\}$ the fix counts $|\mathrm{Fix}(e, \rho, \rho^2, \rho^3)|$ match the closed-form formula exactly (Table 2).
4. **Group count (odd n).** The Burnside count matches the formula $3(2^{2n-4}+2^{n-3}+2^{(n-3)/2})$ at all five odd sizes.
5. **Even n.** For all even $n \leq 10$ the fix counts under non-identity rotations are zero, confirming the general argument. The group count is $|\mathcal{P}_n|/4$.
6. **Set equality (small n).** For $n \leq 7$ the set of proper colourings found by backtracking equals the set produced by `generate_separable`. For $n \geq 8$ set equality is inferred from: (a) every found colouring is separable, and (b) the total count equals $|\mathcal{P}_n|$.

8 Generalisations and Extensions

8.1 General $n \times n$ grids

The analysis applies verbatim to any $n \times n$ grid. The total count is $3 \times 2^{2(n-1)}$. For the Burnside analysis:

Odd $n = 2m + 1$. The grid has a unique centre cell fixed by all four rotations, enabling non-trivial fixed colourings. Our fixed-point analysis gives:

$$|\mathrm{Fix}(\rho^0)| = 3 \cdot 2^{2(n-1)}, \quad |\mathrm{Fix}(\rho^1)| = |\mathrm{Fix}(\rho^3)| = 3 \cdot 2^{(n-1)/2}, \quad |\mathrm{Fix}(\rho^2)| = 3 \cdot 2^{n-1}.$$

The group count is:

$$|\mathcal{C}_n| = \frac{3}{4}\left(2^{2(n-1)} + 2^{n-1} + 2^{(n-1)/2+1}\right) = 3\left(2^{2n-4} + 2^{n-3} + 2^{(n-3)/2}\right).$$

Even n. The $90°$ rotation has no fixed cells; no proper colouring is fixed by ρ or ρ^3, and the same turns out to hold for ρ^2 (by a parallel argument using the impossibility of a palindrome $h_j = h_{n-2-j}$ summing to zero when the pairing forces non-zero partial sums). The group count is simply $|\mathcal{P}_n|/4 = 3 \cdot 2^{2(n-1)-2}$.

Spot-checks. For $n = 3$: $|\mathcal{C}_3| = 3(2^2 + 2^0 + 2^0) = 18$. ✓

For $n = 5$: $|\mathcal{C}_5| = 3(2^6 + 2^2 + 2^1) = 3 \times 70 = 210$. ✓

For $n = 7$: $|\mathcal{C}_7| = 3(2^{10} + 2^4 + 2^2) = 3 \times 1044 = 3132$. ✓

For $n = 9$: $|\mathcal{C}_9| = 3(2^{14} + 2^6 + 2^3) = 3 \times 16456 = 49368$. ✓

For $n = 11$: $|\mathcal{C}_{11}| = 3(2^{18} + 2^8 + 2^4) = 3 \times 262416 = 787248$. ✓

8.2 Rectangular grids

For an $m \times n$ rectangular grid with $m \neq n$, the rotation group is trivial (the $90°$ rotation does not map the grid to itself). Every proper colouring is in its own equivalence class, and the count equals $3 \times 2^{m+n-2}$ (since there are $m - 1$ vertical slopes and $n - 1$ horizontal slopes).

8.3 Other symmetry groups

If one considers the full symmetry group of the square $D_4 = \mathbb{Z}_4 \rtimes \mathbb{Z}_2$ (rotations and reflections), the analysis extends similarly. The reflection $\sigma : (r, c) \mapsto (r, 100 - c)$ (horizontal flip) imposes the constraint $R(c) = R(100-c)+K$ for some constant K, which (by the same argument) forces $K = 0$ and the same anti-palindrome $h_j = h_{99-j}$ with v_r free. This gives $|\mathrm{Fix}(\sigma)| = 3 \times 2^{50} \times 2^{100} = 3 \times 2^{150}$. Burnside's Lemma over D_4 would then yield a smaller count than our $\mathbb{Z}_4$ result.

The key point is that our main formula $3(2^{198} + 2^{98} + 2^{49})$ is computed with $\mathbb{Z}_4$ (the rotation group), which is exactly what the problem specifies.

8.4 More colours

The problem is specific to 3 colours because the richness condition (all colours in every 2×2 square) and the adjacency condition together uniquely force the separable structure. With $k > 3$ colours, one can satisfy the adjacency condition without using all colours in every 2×2 square, and the analysis would differ significantly.

9 Machine-Checked Formal Proof in Lean 4

To provide the highest degree of mathematical certainty, the entire proof has been formalised in **Lean 4** using the MATHLIB library [6]. The formalisation proves, from first principles, that the Burnside orbit count for any odd $n \geq 5$ with $4 \mid (n - 1)$ equals the closed form, and specialises this to $n = 101$.

9.1 Module structure

The Lean library (`GriddlesP1Lean`) is organised into six modules in dependency order:

1. `Defs` — Core types: `Coloring n` (a function $\{0, \ldots, n-1\}^2 \to \mathbb{Z}/3\mathbb{Z}$), the predicates `AdjOk` and `RichOk` encoding the two colouring constraints, the rotation map `rotate90`, and the fixed-point predicates `FixedBy90`/`FixedBy180`.
2. `Separability` — The Separability Theorem (`ProperColoring.isSeparable`): every proper colouring satisfies $f(r, c) = \mathrm{base} + R(c) + D(r)$ for uniquely determined slope sequences.
3. `Slopes` — The nonzero slope type `NZMod3` $= \{1, 2\} \subset \mathbb{Z}/3\mathbb{Z}$ and the palindrome-extending cumulative-sum construction `mkCumSum` used to build the fixed-point bijections.
4. `Bijections` — Three explicit `Equiv`s that parametrise each fixed-point set by a product type:
   - `equiv_proper`: all proper colourings $\longleftrightarrow \mathbb{Z}/3 \times (\mathrm{Fin}(n-1) \to \mathtt{NZMod3})^2$,
   - `equiv_fixed180`: $180°$-fixed colourings $\longleftrightarrow \mathbb{Z}/3 \times (\mathrm{Fin}(m) \to \mathtt{NZMod3})^2$, where $m = (n-1)/2$,
   - `equiv_fixed90`: $90°$-fixed colourings $\longleftrightarrow \mathbb{Z}/3 \times (\mathrm{Fin}(m) \to \mathtt{NZMod3})$.
5. `Counting` — Cardinality results derived from the bijections via `Fintype.card_prod`/`card_pi`:

$$\begin{aligned} \texttt{card\_proper} &: |\mathcal{P}_n| = 3 \cdot 2^{2(n-1)}, \\ \texttt{card\_fixed180\_odd} &: |\mathrm{Fix}(\rho^2)| = 3 \cdot 2^{n-1}, \\ \texttt{card\_fixed90\_odd} &: |\mathrm{Fix}(\rho^1)| = 3 \cdot 2^{(n-1)/2}, \end{aligned}$$

   and the Burnside arithmetic lemma `chromatic_rotations_general` which combines these into the closed form.
6. `Answer` — The general theorem `chromatic_rotations_burnside` and its specialisations to $n = 5, 9, 13, 101$.

9.2 The main theorem

The capstone result, stated and proved in Lean 4:

```
theorem chromatic_rotations_burnside (n : ℕ) (hn : Odd n)
    (hn5 : 5 ≤ n) (h4 : 4 ∣ (n - 1)) :
    (Nat.card {f : Coloring n // AdjOk f ∧ RichOk f}
   + Nat.card {f : Coloring n // AdjOk f ∧ RichOk f ∧ FixedBy90  n f}
   + Nat.card {f : Coloring n // AdjOk f ∧ RichOk f ∧ FixedBy180 n f}
   + Nat.card {f : Coloring n // AdjOk f ∧ RichOk f ∧ FixedBy90  n f}) / 4
  = 3 * (2 ^ (2*(n-1) - 2) + 2 ^ ((n-1)/2 - 1) + 2 ^ (n-3)) := by
  rw [card_proper n (by omega),
      card_fixed180_odd n hn (by omega),
      card_fixed90_odd n hn hn5 h4]
  exact chromatic_rotations_general n hn hn5 h4
```

Listing 3: General Burnside theorem (`Answer.lean`).

The proof rewrites each `Nat.card` term using the proved bijections, then delegates Burnside arithmetic to `chromatic_rotations_general`. The $n = 101$ instance follows by direct application:

```
theorem chromatic_rotations_n101 :
    (...) / 4 = 3 * (2 ^ 198 + 2 ^ 49 + 2 ^ 98) :=
  chromatic_rotations_burnside 101 (by decide) (by norm_num) (by norm_num)
```

Listing 4: The $n = 101$ instance (`Answer.lean`).

The exponents $198 = 2{\cdot}100 - 2$, $49 = 100/2 - 1$, $98 = 101 - 3$ reduce definitionally in Lean's kernel, so no additional arithmetic lemmas are required.

9.3 Proof strategy

The key insight making the formalisation tractable is that *every fixed-point count reduces to a cardinality of a product type*. Rather than counting by inclusion–exclusion or generating functions, each fixed-point set is shown equivalent (via an explicit `Equiv`) to a finite product; MATHLIB then computes the cardinality automatically. The full proof chain from grid definitions to the $n = 101$ answer requires no `sorry`, no `native_decide`, and no unverified arithmetic — every step is either a structural bijection proof or an `omega`/`ring` closure.

Computable formula. As a separate convenience, the library also provides a computable definition:

$$\texttt{chromaticRotations}(n) = 3\left(2^{2n-4} + 2^{(n-3)/2} + 2^{n-3}\right),$$

which can be evaluated with `#eval` for any specific odd n:

n	`chromaticRotations n`
3	18
5	210
7	3,132
9	49,368
13	12,586,080
101	$\approx 1.205 \times 10^{60}$

These values agree with the Python verification in Section 7. The Lean source is available alongside this writeup in `puzzle-1/griddles-p1-lean/`.

Acknowledgements

The mathematical solution to this problem — including the Separability Theorem, the Burnside fixed-point analysis, and the closed-form answer — was developed in collaboration with Claude (Anthropic). The computational verification code, the Lean 4 formalisation, and this writeup were subsequently produced in collaboration with Claude Code (Anthropic).

Computations were performed in Python 3.13 with the `tqdm` library on an Apple M1 Pro laptop (8 CPU cores). The full verification sweep ($n = 2$ through 11) completes in approximately 28 seconds using all 8 cores via Python's `multiprocessing` module, with no GPU or special hardware required. The Lean 4 formalisation uses MATHLIB4 and was type-checked with Lean 4 on the same machine.

References

[1] Burnside, W. (1897). *Theory of Groups of Finite Order*. Cambridge University Press.

[2] Cauchy, A.L. (1845). Mémoire sur diverses propriétés remarquables des substitutions régulières ou irrégulières. *C. R. Acad. Sci. Paris*, **21**, 835–852.

[3] Pólya, G. (1937). Kombinatorische Anzahlbestimmungen für Gruppen, Graphen und chemische Verbindungen. *Acta Math.*, **68**, 145–254.

[4] van Lint, J.H. & Wilson, R.M. (2001). *A Course in Combinatorics* (2nd ed.). Cambridge University Press.

[5] Stanley, R.P. (2011). *Enumerative Combinatorics*, Vol. 1 (2nd ed.). Cambridge University Press.

[6] The Mathlib Community (2020). *Lean 4 Mathematical Library (*MATHLIB4*)*. https://leanprover-community.github.io.

A Worked Examples of Proper Colourings

A.1 The 12 proper colourings of the 2×2 grid

Each proper colouring of the 2×2 grid is determined by choosing a base colour $b \in \{0, 1, 2\}$, a single horizontal slope $h_0 \in \{1, 2\}$, and a single vertical slope $v_0 \in \{1, 2\}$, giving $3 \times 2 \times 2 = 12$ colourings. A representative set is:

Base	h_0	v_0	Grid	Rotation orbit
0	1	1	$\begin{pmatrix} 0 & 1 \\ 1 & 2 \end{pmatrix}$	size 4
0	1	2	$\begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix}$	size 4
0	2	1	$\begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix}$	size 4

(Each orbit has size 4 since no 2×2 proper colouring is fixed by any non-identity rotation, as verified by the script.) The three orbits account for all 12 colourings.

A.2 A separable colouring of the 3×3 grid

Take $\text{base} = 0$, $h = (1, 2)$, $v = (2, 1)$. Then:

$$R = (0, 1, 0), \quad D = (0, 2, 0),$$

and:

$$f = \begin{pmatrix} 0+0+0 & 1+0+0 & 0+0+0 \\ 0+0+2 & 1+0+2 & 0+0+2 \\ 0+0+0 & 1+0+0 & 0+0+0 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 \\ 2 & 0 & 2 \\ 0 & 1 & 0 \end{pmatrix} \pmod 3.$$

Checking: horizontal slopes in each row are $(1, 2)$ as given; vertical slopes in each column are $(2, 1)$ as given. Every 2×2 sub-square contains all three colours. ✓

A.3 A colouring fixed by $180°$ rotation ($n = 3$)

Take $\text{base} = 0$, $h = (1, 2)$ (anti-palindrome: $1 + 2 = 3 \equiv 0$), $v = (2, 1)$ (anti-palindrome: $2 + 1 = 3 \equiv 0$).

$$f = \begin{pmatrix} 0 & 1 & 0 \\ 2 & 0 & 2 \\ 0 & 1 & 0 \end{pmatrix}.$$

Rotating $180°$: $(r, c) \mapsto (2 - r, 2 - c)$:

$$f' = \begin{pmatrix} f(2,2) & f(2,1) & f(2,0) \\ f(1,2) & f(1,1) & f(1,0) \\ f(0,2) & f(0,1) & f(0,0) \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 \\ 2 & 0 & 2 \\ 0 & 1 & 0 \end{pmatrix} = f. ✓$$

The same colouring is also fixed by both $90°$ and $270°$ rotations in this particular case.

B Fixed-Point Constraints: Full Derivation for $n = 101$

B.1 All four cases in full

Below we collect the constraints characterising each fixed-point set for $n = 101$ (indices $0 \dots 100$ for cells, slopes indexed $0 \dots 99$).

$\mathrm{Fix}(\rho^0)$**: identity.** No constraint. Free parameters: $\mathrm{base} \in \{0, 1, 2\}$ (3 choices), $h_j \in \{1, 2\}$ for $j = 0, \dots, 99$ (2^{100} choices), $v_i \in \{1, 2\}$ for $i = 0, \dots, 99$ (2^{100} choices). Total: 3×2^{200}.

$\mathrm{Fix}(\rho^1)$**:** 90° **clockwise.** Constraints: (i) $v_i = h_i$ for all i (no free vertical slopes), (ii) $h_j + h_{99-j} \equiv 0 \pmod 3$ for $j = 0, \dots, 49$ (h_{99-j} determined by h_j). Free: base (3 choices), $h_0, \dots, h_{49}$ (2^{50} choices). Total: 3×2^{50}.

$\mathrm{Fix}(\rho^2)$**:** 180°**.** Constraints: (i) $h_j + h_{99-j} \equiv 0 \pmod 3$ for $j = 0, \dots, 49$, (ii) $v_i + v_{99-i} \equiv 0 \pmod 3$ for $i = 0, \dots, 49$. Free: base (3), $h_0, \dots, h_{49}$ (2^{50}), $v_0, \dots, v_{49}$ (2^{50}). Total: 3×2^{100}.

$\mathrm{Fix}(\rho^3)$**:** 270° **clockwise.** Constraints identical to $\mathrm{Fix}(\rho^1)$ (by symmetry of the derivation under renaming of row/column indices). Total: 3×2^{50}.

B.2 Why the global-sum condition is automatic

In each case, the derivation shows that $D(100) = \sum_{i=0}^{99} v_i \equiv 0 \pmod 3$ is required. Under the anti-palindrome condition $v_i + v_{99-i} \equiv 0$, the sum telescopes:

$$\sum_{i=0}^{99} v_i = \sum_{i=0}^{49} (v_i + v_{99-i}) = \sum_{i=0}^{49} 3 = 150 \equiv 0 \pmod 3.$$

No additional constraint is imposed beyond the pairing, and the choice of 50 free values in $\{1, 2\}$ is unrestricted.